



TP 7 : RNA — Dataset Titanic

Objectifs du TP

- Importer un dataset Kaggle
- Prétraiter un dataset réel (numérique + catégoriel)
- Construire et entraîner un réseau de neurones (Keras/TensorFlow)
- Évaluer, interpréter et améliorer le modèle
- Réaliser des expérimentations sur l'architecture et les hyperparamètres

1. Présentation du Dataset Titanic

Ce dataset contient des informations sur les passagers du Titanic, ainsi que l'indicateur **Survived** (0 = mort, 1 = survécu).

Il comporte des variables :

- **Survived** (cible : 0 = mort, 1 = survécu)
- **Pclass** (classe économique)
- **Sex**
- **Age**
- **SibSp** (nombre de frères/sœurs à bord)
- **Parch** (nombre de parents/enfants à bord)
- **Fare**
- **Embarked** (port d'embarquement)

2. Importation du Dataset

```
!pip install kaggle pandas numpy scikit-learn tensorflow seaborn
```

```
!mkdir ~/.kaggle  
!cp kaggle.json ~/.kaggle/  
!chmod 600 ~/.kaggle/kaggle.json
```

```
!kaggle competitions download -c titanic  
!unzip titanic.zip
```

Chargement :

```
import pandas as pd  
  
train = pd.read_csv('train.csv')  
train.head()
```

3. Prétraitement

Suppression des colonnes non pertinentes :

```
train = train.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])
```

Gestion des valeurs manquantes :

```
train['Age'].fillna(train['Age'].median(), inplace=True)
train['Embarked'].fillna(train['Embarked'].mode()[0], inplace=True)
```

Encodage :

```
train = pd.get_dummies(train, columns=['Sex', 'Embarked'], drop_first=True)
```

4. Split et Normalisation

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X = train.drop("Survived", axis=1)
y = train["Survived"]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

5. Construction du Réseau de Neurones

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.3),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
model.summary()
```

6. Entraînement du Modèle

```
history = model.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=16,  
    validation_split=0.2  
)
```

7. Visualisation des Courbes

```
import matplotlib.pyplot as plt  
  
plt.plot(history.history['loss'], label='loss')  
plt.plot(history.history['val_loss'], label='val_loss')  
plt.legend()  
plt.title("Courbe de Loss")  
plt.show()  
  
plt.plot(history.history['accuracy'], label='accuracy')  
plt.plot(history.history['val_accuracy'], label='val_accuracy')  
plt.legend()  
plt.title("Courbe de Accuracy")  
plt.show()
```

8. Évaluation sur le Test Set

```
model.evaluate(X_test, y_test)
```

9. Prédictions pour un Passager Fictif

```
import numpy as np  
  
nouveau_passager = np.array([[3, 22, 1, 0, 7.25, 0, 1, 0]])  
  
nouveau_passager_scaled = scaler.transform(nouveau_passager)  
prediction = model.predict(nouveau_passager_scaled)  
print("Probabilité de survie :", prediction[0][0])
```

10. QUESTIONS POUR LE COMPTE-RENDU

Questions générales

1. Quelles variables semblent influencer le plus la survie ?
2. Pourquoi est-il nécessaire d'encoder Sex et Embarked ?
3. Quel est l'impact de la normalisation sur la performance du réseau ?

Architecture & Hyperparamètres

4. Comparez les architectures suivantes :
 - une seule couche cachée ;
 - deux couches cachées ;
 - trois couches cachées.Quelle architecture donne les meilleurs résultats ?
5. Testez différentes fonctions d'activation (`relu`, `tanh`, `elu`) et expliquez les différences observées.
6. Comparez les optimizers `SGD`, `Adam` et `RMSprop`.
7. Testez plusieurs taux d'apprentissage (0.1, 0.01, 0.001). Lequel converge le mieux ?

Régularisation & performances

8. Ajoutez du Dropout, puis de la régularisation L2. Comment évolue l'overfitting ?
9. Ajoutez un Batch Normalization : quel est l'effet sur la stabilité et la vitesse d'apprentissage ?

Évaluation approfondie

10. Calculez et interprétez :
 - La matrice de confusion
 - Le score F1
 - La courbe ROC
 - L'AUC
11. Quel seuil de classification optimiserait la précision ou le rappel selon le contexte ?

Comparaison avec d'autres modèles

12. Comparez votre réseau de neurones avec :
 - Un Random Forest
 - Un SVM
 - Un KNNQuel modèle généralise le mieux ? Expliquez pourquoi.

Analyse & interprétation

13. Utilisez SHAP ou permutation importance pour interpréter l'importance des variables.
14. Expliquez en vos mots le fonctionnement de la rétropropagation.
15. Que se passe-t-il si vous entraînez trop longtemps ou pas assez longtemps ?

Noté bien : C'est une méthode d'interprétation des modèles d'intelligence artificielle qui permet de comprendre comment chaque variable contribue à la prédiction d'un modèle (RNA, Random Forest, XGBoost, etc.).

Alors :

SHAP explique "pourquoi" le modèle a fait une prédiction.

1) L'idée générale :

SHAP repose sur une théorie mathématique appelée **valeurs de Shapley**, issue de la théorie des jeux coopératifs.

Imagine :

- Chaque **variable** = un joueur.
- La **prédiction du modèle** = un gain.
- SHAP calcule combien **chaque variable** “a contribué” au résultat.

2) À quoi sert SHAP ?

SHAP sert à :

✓ Comprendre une prédiction individuelle

Exemple :

Pourquoi l'algorithme a prédit que ce passager du Titanic n'a pas survécu ?

SHAP dira :

- Sexe = homme : -0.28
- Age = 45 ans : -0.12
- Classe = 3 : -0.24
- Tarif = élevé : +0.05
- etc.

✓ Visualiser l'importance globale des variables

SHAP peut produire un graphique montrant les variables les plus influentes.

✓ Détecter des biais

Par exemple : le modèle s'appuie trop sur le sexe ou la classe sociale.

✓ Expliquer les modèles “boîte noire”

CNN, réseaux denses, Random Forests, Gradient Boosting, etc.